

Sequence Modeling: RNNs and the Temporal Dimension

Scaler School of Technology | Deep Learning I | Week 5 — Session 2/2 | Post-Lecture Review & Practice



Key Takeaways

1 RNNs Process Sequences via Weight Sharing

Unlike MLPs, RNNs apply the **same weights at every timestep**, maintaining a hidden state h_t that summarizes all past inputs. This handles variable-length sequences with a fixed number of parameters.

2 BPTT Unrolls the Loop for Training

Backpropagation Through Time treats the unrolled RNN as a deep feedforward network. The gradient of the loss w.r.t. weights involves multiplying Jacobians across all timesteps — making it a product of many matrices.

3 Vanishing Gradients Limit Long-Range Learning

When the spectral radius of W_{hh} is < 1 , gradients decay exponentially over time. The RNN effectively “forgets” inputs from more than ~ 20 steps ago. This is the core motivation for LSTMs and GRUs (next week).

Test Your Understanding

Try answering these without looking at the slides. Answers are on page 3.

- Q1.** Why can't a standard MLP handle sequences of different lengths without modification?
- Q2.** Write the vanilla RNN recurrence equation. What are the three weight matrices and what does each connect?
- Q3.** Explain in one sentence why the unrolled RNN resembles a very deep feedforward network.
- Q4.** If the largest singular value of W_{hh} is 0.9 and your sequence is 50 timesteps long, approximately how much of the gradient from timestep 50 reaches timestep 1?
- Q5.** Name two tasks that use a many-to-one RNN configuration and one task that uses many-to-many.

Work through these in a Colab notebook. No starter code — figure out the approach from what you learned in the lecture.

Exercise 1: Implement a Vanilla RNN from Scratch

Using only numpy, implement a vanilla RNN class with `forward()` that processes a sequence of inputs. Test it on a random sequence of length 20 with `input_size=5` and `hidden_size=16`. Print the hidden state norm at each timestep and plot it. **Expected output:** A line plot showing how the hidden state magnitude evolves over time.

Exercise 2: Vanishing Gradient Experiment

Using PyTorch, create an RNN processing sequences of length 10, 30, 50, and 100. For each, compute the loss using only the final hidden state, backpropagate, and record the gradient magnitude at the **first** timestep. Plot gradient magnitude vs. sequence length. **Expected output:** A bar chart showing exponential gradient decay with sequence length.

Exercise 3: Sequence Classification

Build a many-to-one RNN that classifies binary sequences (length 10–20) as “majority ones” or “majority zeros”. Generate 1000 random sequences, train for 50 epochs, and report test accuracy. **Expected output:** Training loss curve + test accuracy > 90%.

Challenge Problem

Character-Level Name Generator

Build a character-level RNN that generates realistic-sounding names after training on a dataset of names.

1. Download a names dataset (e.g., a list of 1000+ first names)
2. One-hot encode characters and create input/target pairs (input = char t , target = char $t + 1$)
3. Build and train a single-layer RNN (`hidden_size=128`) with cross-entropy loss
4. Implement a sampling function: start with a random character, feed predictions back as input
5. Generate 10 names and evaluate if they look realistic

Bonus: Add a “temperature” parameter to your sampling function. Show how temperature=0.5 vs. 1.0 vs. 1.5 affects the diversity and quality of generated names.

- **RNN Foundations:** Elman (1990), *“Finding Structure in Time”* — The original paper introducing simple recurrent networks. Short and readable.
- **Vanishing Gradients:** Bengio et al. (1994), *“Learning Long-Term Dependencies with Gradient Descent is Difficult”* — The paper that formally characterized why RNNs fail on long sequences.
- **Practical Guide:** Karpathy (2015), *“The Unreasonable Effectiveness of Recurrent Neural Networks”* — A blog post with beautiful visualizations of character-level RNNs and hidden state dynamics.
- **BPTT Details:** Werbos (1990), *“Backpropagation Through Time: What It Does and How to Do It”* — The definitive reference for BPTT derivation.

Online Resources & Tools

- **PyTorch nn.RNN docs** — Official API reference with examples
- **Colah’s blog: “Understanding LSTMs”** — Excellent diagrams of RNN/LSTM internals
- **Stanford CS231n RNN notes** — Clear derivation of BPTT
- **D2L.ai Ch. 9** — Interactive RNN chapter with runnable code
- **Jay Alammar’s blog** — Visual guides to sequence models
- **3Blue1Brown: Neural Networks** — Video intuition for backprop and gradients

Fill the Gaps — Prerequisite Refreshers

Felt lost on a specific concept? These resources will help you catch up.

- **Confused about the chain rule through time?** Revisit the Week 2 backpropagation lecture and computational graphs. The same chain rule applies here — just repeated across timesteps instead of layers.
- **Fuzzy on matrix eigenvalues / singular values?** Watch 3Blue1Brown’s “Essence of Linear Algebra” (Chapters 7–8). You only need the intuition that eigenvalues tell you if a matrix shrinks or grows vectors.
- **Not sure how tanh works as an activation?** Recall from Week 1 Session 2: tanh squashes values to $[-1, 1]$ and its derivative is always ≤ 1 . This is why repeated application causes vanishing.
- **Struggling with PyTorch tensor shapes?** Practice with the official PyTorch “60-Minute Blitz” tutorial. RNNs need inputs shaped as (batch, sequence_length, features).

Answers — Test Your Understanding

- A1.** An MLP requires a fixed-size input vector. Sequences have variable lengths, so you’d need to pad to the maximum length — wasting parameters and losing the notion of temporal order.
- A2.** $h_t = \tanh(W_{xh} \cdot x_t + W_{hh} \cdot h_{t-1} + b)$. W_{xh} : input→hidden, W_{hh} : hidden→hidden (memory update), W_{hy} : hidden→output. Three matrices, shared across all timesteps.
- A3.** Each timestep in the unrolled RNN acts like one layer in a feedforward net — so a 100-step sequence becomes equivalent to a 100-layer network during backprop.
- A4.** $0.9^{50} \approx 0.005$, so only about 0.5% of the gradient reaches timestep 1. The network effectively cannot learn from that early input.
- A5.** Many-to-one: sentiment analysis, document classification. Many-to-many: POS tagging, machine translation, named entity recognition.